

```
# body of function

# ... more functions

def <functionNameN>(self, <optional param1>, ..., <optional paramN>):
    # body of function
```

Listing 2-2: The typical form of a class in Python

You begin a class definition with a `class` statement specifying the name you want to give the class. The convention for class names is to use camel case, with the first letter uppercase (for example, `LightSwitch`). Following the name you can optionally add a set of parentheses, but the statement must end with a colon to indicate that you're about to begin the body of the class. (I'll explain what can go inside the parentheses in Chapter 10, when we discuss inheritance.)

Within the body of the class, you can define any number of functions. All the functions are considered part of the class, and the code that defines them must be indented. Each function represents some behavior that an object created from the class can perform. All functions must have at least one parameter, which by convention is named `self` (I'll explain what this name means in Chapter 3). OOP functions are given a special name: *method*.

method

A function defined inside a class. A method always has at least one parameter, which is usually named `self`.

The first method in every class should have the special name `__init__`. Whenever you create an object from a class, this method will run automatically. Therefore, this method is the logical place to put any initialization code that you want to run whenever you instantiate an object from a class. The name `__init__` is reserved by Python for this very task, and it must be written exactly this way, with two underscores before and after the word *init* (which must be lowercase). In reality, the `__init__()` method is not strictly required. However, it's generally considered good practice to include it and use it for initialization.